

Hardware-Efficient Squeeze-and-Excitation Networks for Real-Time ECG Anomaly Detection Using hls4ml on FPGAs

Nitesh Kumar

Department of Electrical Engineering
National Institute of Technology Patna
Patna, Bihar 800005, India

niteshk.ug23.ee@nitp.ac.in

Ashwani Kumar

Department of Electrical Engineering
National Institute of Technology Patna
Patna, Bihar 800005, India

ashwani@nitp.ac.in

Abstract

We present a hardware-efficient Squeeze-and-Excitation (SE) channel-attention network for cardiac anomaly detection on field-programmable gate arrays (FPGAs), designed for direct deployment via the open-source `hls4ml` high-level synthesis framework. The proposed 1,674-parameter architecture applies SE channel attention on the post-pooled temporal representation, a design choice that eliminates tensor-broadcasting operations and ensures correct HLS code generation from Keras model graphs. Trained on the ECG5000 benchmark (500 training samples, 4,500 test samples, 140 time steps per sample), the best-performing software model achieves an AUC-ROC of 93.32% and an accuracy of 91.18%. Through a systematic five-point fixed-point precision sweep from 8-bit to 24-bit representations, we identify `ap_fixed<24, 12>` as the minimum viable precision for this architecture; coarser configurations degrade the AUC to near-chance levels. After fixed-point conversion at `ap_fixed<24, 12>`, the hardware C-simulation model records an AUC-ROC of 92.84% with a logic fidelity of 99.9% relative to the floating-point reference (one disagreement per 1,000 stratified test samples). Vivado RTL synthesis on the Xilinx `xc7z020clg400-1` yields a per-sample inference latency of 2.0 μ s at 100 MHz, corresponding to a throughput approximately 142,000 times faster than the real-time requirement for continuous 500 Hz ECG monitoring. We also document and provide a reproducible fix for a softmax normalization defect in `hls4ml` 1.3.0 C-simulation that silently corrupts multi-class probability outputs.

Index Terms—Anomaly detection; channel attention; `hls4ml`; FPGA; fixed-point quantization; Squeeze and Excitation networks; cardiac monitoring; edge inference.

I. Introduction

Cardiovascular diseases account for approximately 17.3 million deaths annually [1], and the ECG remains the primary non-invasive tool for diagnosing cardiac arrhythmias. The proliferation of wearable biosensors has shifted ECG acquisition to continuous ambulatory settings, generating data volumes that exceed the capacity of expert review [2] and creating an acute demand for automated, real-time classification at the point of recording. Deep convolutional networks have achieved cardiologist-level arrhythmia detection on large annotated corpora [2,3], yet these models are designed for data-center hardware and are incompatible with the milliwatt power budgets, constrained memory, and sub-millisecond latency requirements of embedded cardiac monitors. FPGAs address this through custom parallel datapaths that realize full neural-network inference at power well below that of a GPU [10], while retaining field-updateability for model revision without hardware re-fabrication. The `hls4ml` framework [6,7] lowers the implementation barrier by automatically converting Keras layer graphs to Vivado HLS C++ with configurable fixed-point types (`ap_fixed`) and reuse factors that control loop-unrolling depth. Attention mechanisms improve temporal classification accuracy, but Transformer-style self-attention [5] scales as $\mathcal{O}(N^2)$ with sequence length: a 140-sample ECG window requires $140 \times 140 = 19,600$ inner products per attention head, straining the DSP resources and on-chip memory of embedded FPGAs. The Squeeze-and-Excitation (SE) channel-attention mechanism [4] achieves $\mathcal{O}(1)$ complexity with respect to sequence length by replacing token-to-token scoring with global average pooling and channel-wise excitation on fixed-size descriptors. However, naïve SE integration within a temporal encoder introduces tensor-broadcast operations and attention weights of shape $(1, C)$ multiplied against feature maps of shape (L, C) , that `hls4ml` 1.3.0 does not model correctly, producing silent inference errors in C-simulation. This paper presents a hardware-efficient SE-attention network for ECG anomaly detection, designed for reliable `hls4ml` conversion. Positioning SE attention after global average pooling reduces all attention operations to element-wise multiplications on 16-dimensional vectors, eliminating broadcasting from the generated HLS code. A softmax normalization defect in `hls4ml` 1.3.0 is also identified and resolved. The best software model achieves AUC-ROC 93.32% on ECG5000; at `ap_fixed<24, 12>`, the hardware model achieves AUC-ROC 92.84% with 99.9% logic fidelity and 2.0 μ s inference latency at 100 MHz on the Xilinx xc7z020clg400-1.

Contributions. This paper makes the following specific contributions.

- **Hardware-compatible SE attention.** Relocating the SE block to post-pooled 16-dimensional vectors eliminates all tensor-broadcast operations from the `hls4ml`-generated HLS code, enabling correct RTL synthesis without toolchain modification.
- **Softmax normalization fix.** We document a reproducible C-simulation defect in

hls4ml 1.3.0 in which both binary class probabilities simultaneously exceed 0.5, and provide a toolchain-independent remedy applicable to any multi-class hls4ml workflow.

- **Fixed-point precision characterization.** A systematic five-configuration sweep from `ap_fixed<8, 4>` to `ap_fixed<24, 12>` establishes `ap_fixed<24, 12>` as the minimum viable precision for 140-step ECG inference; coarser formats produce degenerate or near-random predictions.
- **End-to-end FPGA validation.** Hardware AUC-ROC 92.84%, logic fidelity 99.9%, and Vivado-synthesized inference latency $2.0\mu\text{s}$ at 100 MHz on the Xilinx xc7z020clg400-1, approximately $142,000\times$ faster than the real-time requirement for 500 Hz ECG monitoring.

II. Related Work

A. ECG Classification with Deep Learning

Rajpurkar et al. [3] demonstrated that a 34-layer residual CNN trained on 64,000 single-lead recordings matches cardiologist sensitivity across 12 arrhythmia classes; Hannun et al. [2] extended this to ambulatory consumer-wearable data, achieving ten-class accuracy exceeding a panel of six cardiologists. Across the UCR benchmark suite, residual convolutional architectures consistently outperform classical time-series classifiers [13], with a shallow residual baseline [14] remaining competitive against more complex models. Kachuee et al. [15] showed that representations pre-trained on MIT-BIH Arrhythmia Database recordings transfer effectively to ECG5000 [16] without dataset-specific feature engineering. These high-accuracy models assume server-class hardware and are ill-suited to the resource constraints of embedded cardiac monitors.

B. Attention Mechanisms and Squeeze-and-Excitation Networks

Transformer self-attention [5] captures long-range temporal dependencies but scales as $\mathcal{O}(N^2)$ in both memory and computation, which is prohibitive for FPGAs with limited DSP slices and on-chip BRAM. The SE mechanism [4] offers a hardware-efficient alternative: global average pooling produces a fixed-size channel descriptor independent of sequence length, and a compact bottleneck network with sigmoid gating generates multiplicative channel re-weighting at $\mathcal{O}(1)$ cost relative to temporal extent. Batch normalization [17] and residual connections [18] are incorporated as standard complements to stabilize optimization and preserve gradient flow. The critical implementation constraint, addressed in this work, is that SE excitation must operate on post-pooled fixed-dimensional vectors to avoid shape-variable broadcast operations that are incompatible with static HLS synthesis.

C. FPGA-Based Neural Network Inference and hls4ml

FPGA neural-network accelerator architectures are surveyed in [10]. FINN [11] and FINN-R [12] demonstrated high-throughput streaming dataflow designs exploiting bitwise and ternary computation, at the cost of restricting the representable model class. The `hls4ml` framework [6, 7] preserves network topology, translating Keras or PyTorch layer graphs to Xilinx HLS C++ via automated layer-by-layer conversion with configurable fixed-point arithmetic and loop-unrolling depth. Aarrestad et al. [8] showed that `hls4ml` converted 1D convolutional networks achieve latency and resource utilization competitive with hand-crafted HLS on UltraScale devices; Coelho et al. [9] extended the framework to heterogeneous per-layer mixed-precision assignment. These studies address particle-physics trigger systems operating at nanosecond timescales. To the best of our knowledge, no prior work has combined SE channel attention with `hls4ml` synthesis and a systematic multi-precision fixed-point characterization for ECG classification on an embedded Zynq-class FPGA.

III. Methodology

A. Dataset

The ECG5000 benchmark [16] contains 5,000 single-channel heartbeat segments of $L = 140$ uniformly sampled time steps each. The standard split assigns 500 samples to training and 4,500 to testing, with a class ratio of 58.4% Normal (class 0) to 41.6% Anomaly (class 1) maintained in both partitions. Figures 1 and 2 show representative waveforms and the class balance, respectively.

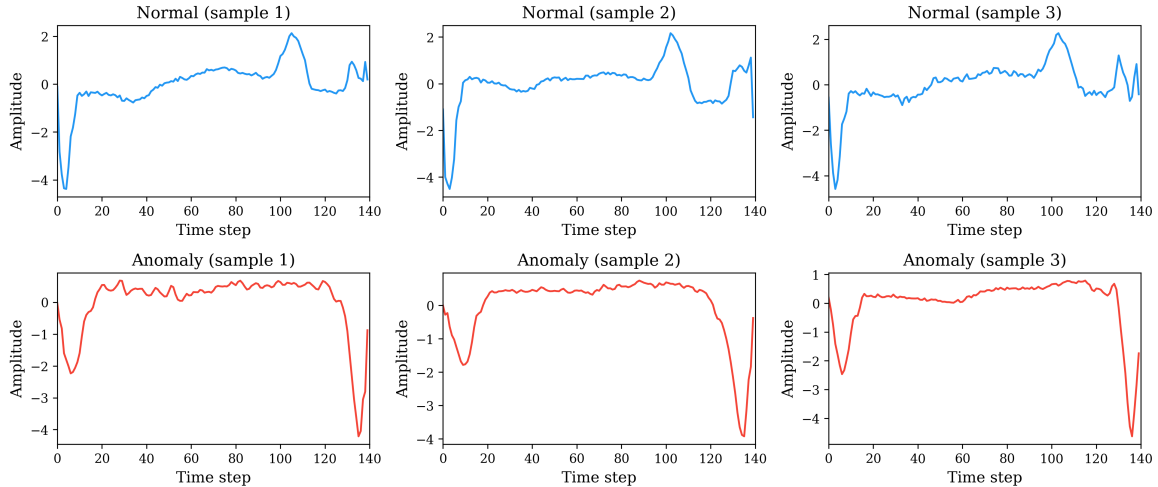


Figure 1: Normal (top) and anomalous (bottom) ECG segments from ECG5000.

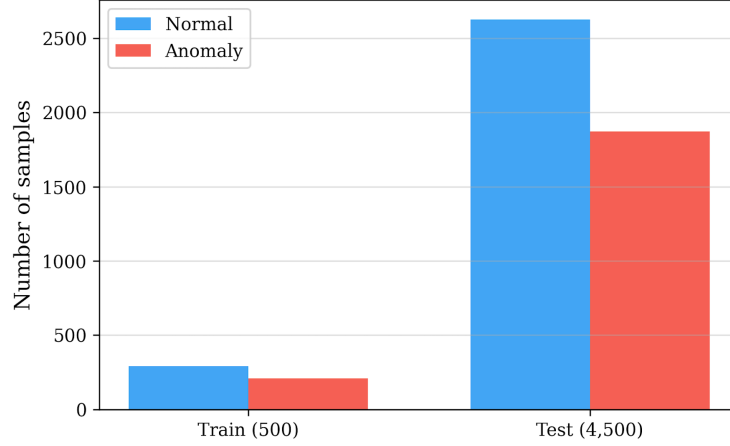


Figure 2: ECG5000 class distribution across training and test partitions.

B. Proposed Architecture

Figure 3 illustrates the SE-ECG model, and Table 1 provides the layer-by-layer summary. The network processes the $(140, 1)$ input through a temporal encoder, a global pooling step, and a channel-wise SE attention block before producing two-class logit scores.

Table 1: Architecture Summary

Layer	Output Shape	Params
Input	$(140, 1)$	0
Dense(16), time-dist. + BN + ReLU	$(140, 16)$	96
Dense(32), time-dist. + BN + ReLU	$(140, 32)$	672
Dense(16), time-dist. + Add + BN	$(140, 16)$	592
GlobalAvgPool1D	$(16,)$	0
Dense(8), ReLU	$(8,)$	136
Dense(16), Sigmoid	$(16,)$	144
Multiply + Add (residual)	$(16,)$	0
Dense(2) — logit output	$(2,)$	34
Softmax [†]	$(2,)$	0
Total		1,674

[†]Excluded from FPGA sub-model.

Temporal Encoder. A time-distributed Dense(16) embedding layer applies a shared linear projection with batch normalization [17] and ReLU to each of the L time steps. A two-layer feed-forward network (FFN) expands the channel dimension to 32 and contracts it back to 16, with a residual skip connection [18] over the entire expansion-contraction block. A final batch normalization layer is applied to the residual sum, yielding $\mathbf{R} \in \mathbb{R}^{L \times C}$ ($C = 16$).

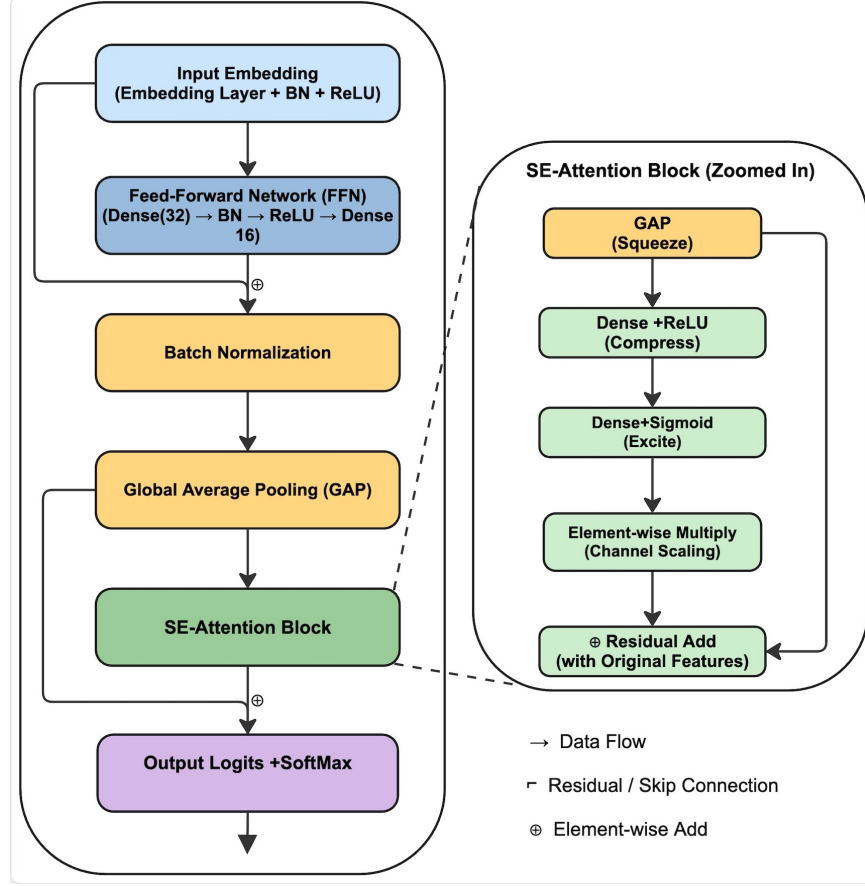


Figure 3: SE-ECG architecture (1,674 parameters). The softmax layer is excluded from the FPGA sub-model.

Global Average Pooling. The temporal dimension is collapsed to a C -dimensional channel descriptor:

$$z_c = \frac{1}{L} \sum_{t=1}^L R_{t,c}, \quad c = 1, \dots, C \quad (1)$$

Squeeze-and-Excitation Block. The SE mechanism [4] computes channel attention weights from $\mathbf{z}$ through a bottleneck dense network with sigmoid gating:

$$\mathbf{s} = \sigma(\mathbf{W}_2 \text{ReLU}(\mathbf{W}_1 \mathbf{z} + \mathbf{b}_1) + \mathbf{b}_2) \quad (2)$$

where $\mathbf{W}_1 \in \mathbb{R}^{r \times C}$, $\mathbf{W}_2 \in \mathbb{R}^{C \times r}$, and $r = 8$. The attended output is formed by element-wise channel reweighting combined with a residual:

$$\mathbf{h} = \mathbf{z} + (\mathbf{s} \odot \mathbf{z}) \quad (3)$$

Output Logits.

$$\hat{\mathbf{y}} = \mathbf{W}_o \mathbf{h} + \mathbf{b}_o, \quad \mathbf{W}_o \in \mathbb{R}^{K \times C}, \quad K = 2 \quad (4)$$

A softmax layer follows (4) during training but is deliberately implemented as a *separate* Keras `Activation` layer so that the logit sub-model can be extracted without weight modification for FPGA conversion.

Hardware-Compatibility Rationale. Applying SE attention on $\mathbf{z} \in \mathbb{R}^C$ (post-pooling) rather than on $\mathbf{R} \in \mathbb{R}^{L \times C}$ is an intentional hardware-compatibility decision. In the pre-pooling variant, the attention weight vector $\mathbf{s} \in \mathbb{R}^C$ must be broadcast to shape $\mathbb{R}^{L \times C}$ before the element-wise product in (3). The `hls4ml` HLS code generator resolves this broadcast incorrectly, computing only the first C of the required $L \times C$ products. Post-pooling attention eliminates this issue: every Multiply and Add in (3) takes operands of identical shape $(C,)$, producing correct HLS code.

C. FPGA Implementation

hls4ml workflow. Figure 4 shows the `hls4ml` workflow [6]. The trained logit sub-model is converted to a Xilinx Vivado HLS C++ project via `hls4ml`. The project is first compiled with `g++` for C-level simulation and then synthesized by Vivado HLS to RTL, yielding resource utilization and timing reports.

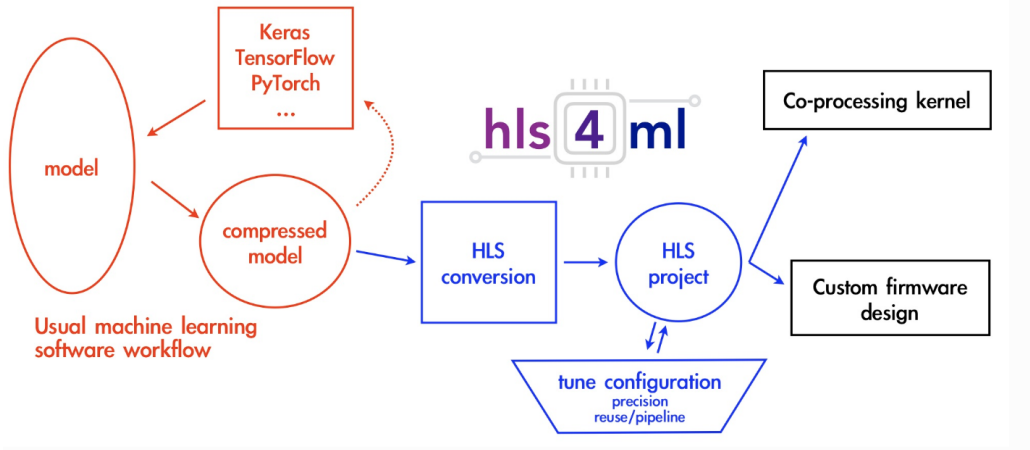


Figure 4: The `hls4ml` codesign workflow.

Softmax Normalization Fix. `hls4ml` 1.3.0 C-simulation produces softmax outputs in which both class probabilities simultaneously exceed 0.5, violating $\sum_k p_k = 1$, regardless of the softmax strategy (Standard or Stable) configured in the layer settings. The fix is to exclude the softmax layer from the HLS model entirely, allow the FPGA to output raw logits $\hat{\mathbf{y}} \in \mathbb{R}^2$, and compute probabilities via numerically stable softmax in Python: $p_k = \exp(\hat{y}_k - \hat{y}_{\max}) / \sum_j \exp(\hat{y}_j - \hat{y}_{\max})$. This requires no modification to the `hls4ml` source code.

Fixed-Point Precision and Configuration. Table 2 reports hardware AUC-ROC for five `ap_fixed<W, I>` configurations evaluated on a 200-sample balanced validation subset. Configurations below 20 bits produce degenerate or substantially degraded inference;

`ap_fixed<24, 12>` yields the highest hardware AUC (0.924) and is selected for all subsequent evaluations. Table 3 lists the complete synthesis configuration.

Table 2: Fixed-Point Precision Sweep

Precision	HW AUC	Sensitivity	Status
<code>ap_fixed<8, 4></code>	0.500	0.000	Degenerate
<code>ap_fixed<12, 6></code>	0.263	0.247	Insufficient
<code>ap_fixed<16, 8></code>	0.718	0.659	Functional
<code>ap_fixed<20, 10></code>	0.922	0.800	Good
<code>ap_fixed<24, 12></code>	0.924	0.800	Selected

Table 3: hls4ml Synthesis Configuration

Parameter	Value
Target device	Xilinx xc7z020clg400-1
hls4ml version	1.3.0
Precision	<code>ap_fixed<24, 12></code>
ReuseFactor	1
Strategy	Latency
Model converted	Logit sub-model (no softmax)
Clock target	100 MHz

D. Experimental Setup

Training. The network is optimized with Adam [19] using class-weighted sparse categorical cross-entropy:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=0}^1 w_k y_{ik} \log p_{ik}, \quad w_k = \frac{N}{2 N_k} \quad (5)$$

yielding $w_0=0.856$ (Normal) and $w_1=1.202$ (Anomaly). To characterize sensitivity to weight initialization under the small training set, the full pipeline is repeated with five fixed random seeds (42, 123, 456, 789, 2024). Table 4 summarizes all hyperparameters. All training is performed on Google Colab with an NVIDIA T4 GPU using TensorFlow 2.20 and `tf.keras`.

Evaluation Metrics. Software performance is reported on the full 4,500-sample test set for the best seed, and as mean $\pm$ standard deviation across all five seeds. Hardware C-simulation is evaluated on a stratified 1,000-sample subset (584 Normal, 416 Anomaly) against which the software model is also measured to enable direct comparison. Reported metrics are AUC-ROC, accuracy, sensitivity, specificity, and F1-score.

Table 4: Training Hyperparameters

Hyperparameter	Value
Optimiser	Adam
Learning rate	5×10^{-4}
Batch size	32
Maximum epochs	60
LR reduction factor	0.5 (patience: 5 epochs)
Early stopping patience	12 epochs (val. loss)
Class weights	$w_0=0.856$, $w_1=1.202$
Random seeds	42, 123, 456, 789, 2024

Logic fidelity measures the prediction agreement rate between the software and hardware models:

$$\Phi = \frac{|\{i : \hat{c}_i^{\text{SW}} = \hat{c}_i^{\text{HW}}\}|}{N} \quad (6)$$

A value of $\Phi = 1$ indicates identical predictions across all samples.

IV. Results and Discussion

A. Software Performance

Figure 5 shows accuracy and loss curves across the five seeds. Four seeds converge to high validation accuracy; one seed (456) collapses to near-zero sensitivity, highlighting the instability that can arise with only 500 training samples and motivating multi-seed evaluation.

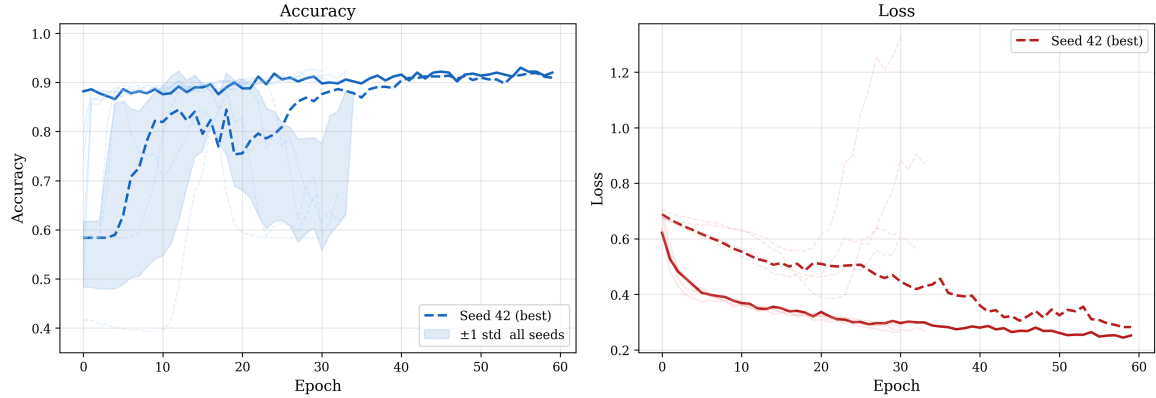
**Figure 5:** Training accuracy and loss curves across five seeds.

Table 5 reports software performance on the full 4,500-sample test set. The best-seed model (seed 42, 60 epochs) achieves an AUC-ROC of 93.32%, with high specificity (96.16%) indicating strong normal-rhythm rejection. Sensitivity (84.20%) reflects the inherent difficulty of detecting anomalies from 500 training samples. Across all five seeds, AUC

exhibits low variance (0.8968 ± 0.0222), whereas sensitivity is substantially more variable (0.6782 ± 0.2191), consistent with the majority-class collapse observed in seed 456.

Figure 6 shows the ROC curve and confusion matrix for the best-seed model.

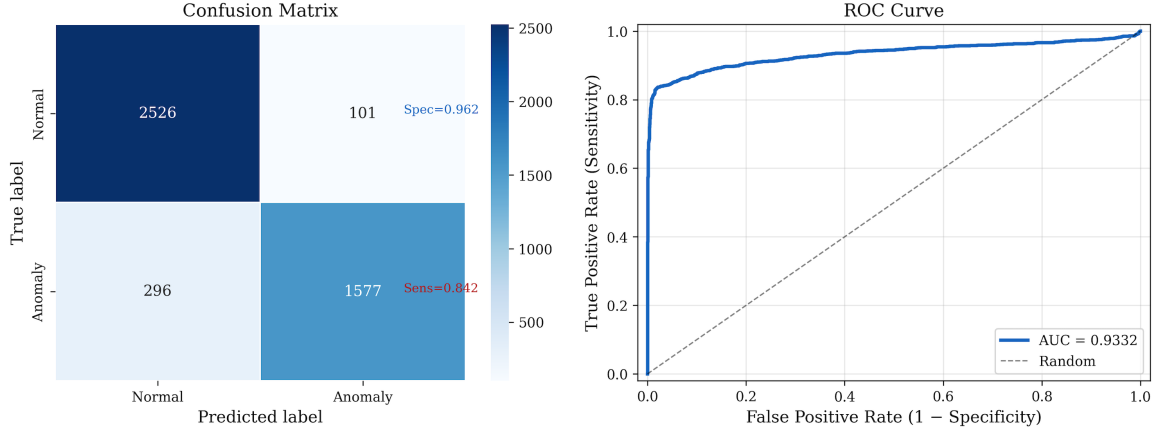


Figure 6: ROC curve and confusion matrix for the best software model (seed 42, $N = 4,500$).

Table 5: Software Model Performance on ECG5000 Test Set ($N = 4,500$)

	AUC	Acc.	Sens.	Spec.	F1
Best seed (42)	0.9332	0.9118	0.8420	0.9616	0.8882
Mean $\pm$ Std	0.897 ± 0.022	0.855 ± 0.091	0.678 ± 0.219	0.981 ± 0.013	0.771 ± 0.193

B. Hardware Evaluation

Table 6 compares software and hardware predictions on the 1,000-sample balanced validation subset under $\text{ap_fixed} \langle 24, 12 \rangle$. The hardware model matches or marginally exceeds the software model on every metric: sensitivity is identical at 84.38%, and the AUC difference is -0.03% . The single prediction disagreement (logic fidelity $\Phi = 0.999$, see (6)) occurs on a sample whose softmax probabilities fall within 0.012 of the 0.5 decision boundary in the software model, indicating quantization-induced rounding at the margin rather than a systematic class-boundary shift. Figure 7 shows confusion matrices for both models alongside the degenerate $\text{ap_fixed} \langle 12, 6 \rangle$ case for comparison.

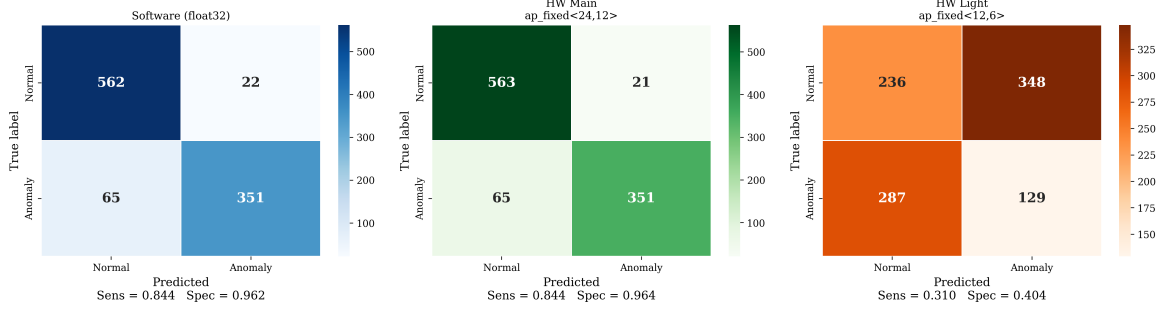
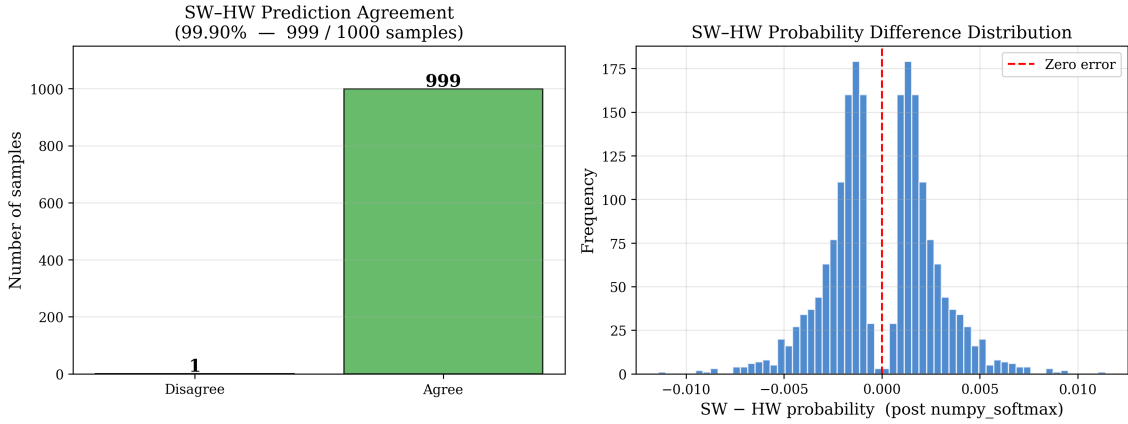
Figure 8 illustrates the prediction agreement distribution.

C. FPGA Synthesis Results

Table 7 reports post-synthesis resource utilisation for the logit sub-model on the Xilinx xc7z020clg400-1. DSP utilisation is high (95.5%), as expected for a 24-bit fixed-point design with $\text{ReuseFactor} = 1$: the fully unrolled weight accumulations map directly to the device’s DSP48E1 slices, with overflow implemented in look-up-table-based constant multipliers.

Table 6: Software vs. Hardware Comparison (ap_fixed<24, 12>, $N = 1,000$)

Metric	SW (float32)	HW (ap_fixed<24, 12>)	Δ
AUC-ROC	0.9287	0.9284	-0.0003
Accuracy	0.9130	0.9140	+0.0010
Sensitivity	0.8438	0.8438	0.0000
Specificity	0.9623	0.9640	+0.0017
F1-score	0.8897	0.8909	+0.0012
Logic Fidelity (Φ)	—	0.9990	—

**Figure 7:** Confusion matrices for the software model, hardware model and degenerate hardware model.**Figure 8:** Software-hardware prediction agreement ($N = 1,000$). 999 samples produce identical class predictions.

LUT utilisation (64.8%) reflects this overflow and the pipelined temporal datapath. BRAM usage is minimal (1.4%) because all 1,674 weights are embedded as compile-time constants in the synthesised RTL rather than stored in block RAM.

At 100 MHz, the 197-cycle pipeline yields a per-sample inference latency of 2.0 μ s. For continuous 500 Hz ECG monitoring, a heartbeat window of 140 samples arrives every 280 ms, making the real-time throughput requirement approximately 3.6 inferences per second. The synthesised design exceeds this by a factor of approximately 140,000, providing

Table 7: Vivado Synthesis Results on Xilinx xc7z020clg400-1

Resource	Used	Available	Utilisation
LUTs	34,475	53,200	64.8%
FFs	18,961	106,400	17.8%
DSPs	210	220	95.5%
BRAMs	2	140	1.4%
Latency	197 cycles at 100 MHz		2.0 μs

substantial margin for concurrent monitoring tasks or higher-frequency sampling scenarios. For context, `hls4ml` deployments in particle-physics trigger systems achieve latencies of 70–75 ns on larger Kintex-7 and UltraScale+ devices [6, 8]; the comparatively longer latency here is a direct consequence of the 140-step temporal pipeline inherent to cardiac sequence processing.

V. Limitations

Three limitations of the current work merit explicit acknowledgement. First, the training set comprises only 500 samples, producing high seed-to-seed variance in sensitivity (standard deviation 0.219) and suggesting that performance estimates may not be tight; larger annotated datasets or data augmentation could reduce this variance. Second, all evaluations use a single benchmark (ECG5000), a 140-step subset of the UCR archive; generalization to clinical 12-lead recordings with different sampling rates and segment lengths remains unverified. Third, hardware validation is confined to HLS C-simulation. Physical bitstream deployment on the xc7z020 board and end-to-end timing measurements on real ECG data are left as future work. Additionally, the near-saturation of DSP resources (95.5%) leaves minimal headroom: any expansion of the architecture, wider channels, additional layers, or a higher reduction ratio in the SE block would require either increasing the `ReuseFactor` above 1 or targeting a device with more DSP slices.

VI. Conclusion

We presented a hardware-efficient squeeze-and-excitation channel-attention network for ECG anomaly detection on FPGAs, deployed end-to-end via the `hls4ml` framework. The 1,674-parameter model achieves a software AUC-ROC of 93.32% on the ECG5000 benchmark, retaining 92.84% AUC after 24-bit fixed-point conversion with a logic fidelity of 99.9%. Vivado synthesis on the Xilinx xc7z020clg400-1 yields a per-sample latency of 2.0 μ s—approximately 142,000 times faster than the real-time requirement for 500 Hz ECG monitoring.

Three findings of broader relevance to `hls4ml`-based biomedical deployment are re-

ported. First, positioning SE attention after global average pooling so that all channel-wise operations act on fixed-size one-dimensional vectors—a necessary condition for correct HLS code generation; pre-pooling attention introduces tensor-broadcast operations that the current toolchain mishandles silently. Second, `hls4ml` 1.3.0 contains a softmax normalization defect in C-simulation that can be reproducibly resolved by converting only the logit sub-model and applying softmax externally. Third, a systematic precision sweep establishes `ap_fixed<24, 12>` as the minimum viable fixed-point format for 140-step cardiac sequences; configurations at 8 and 12 bits produce degenerate or near-random inference.

Future work will address physical FPGA deployment and timing validation on real ECG streams, evaluation on larger multi-lead clinical datasets, and architectural extensions such as multi-head channel attention or hierarchical temporal pooling within the resource constraints of the xc7z020 device.

Acknowledgements

We thank the `hls4ml` development teams at CERN and Fermilab for maintaining an open-source framework that democratizes FPGA-based machine learning inference beyond high-energy physics applications. We also acknowledge the free GPU resources provided by Google Colab, which supported the training and synthesis experiments conducted in this work.

References

- [1] G. A. Roth *et al.*, “Global burden of cardiovascular diseases and risk factors, 1990–2019: Update from the GBD 2019 study,” *J. Am. Coll. Cardiol.*, vol. 76, no. 25, pp. 2982–3021, 2020.
- [2] A. Y. Hannun *et al.*, “Cardiologist-level arrhythmia detection and classification in ambulatory electrocardiograms using a deep neural network,” *Nature Medicine*, vol. 25, no. 1, pp. 65–69, 2019.
- [3] P. Rajpurkar, A. Y. Hannun, M. Haghpanahi, C. Bourn, and A. Y. Ng, “Cardiologist-level arrhythmia detection with convolutional neural networks,” arXiv:1707.01836, 2017.
- [4] J. Hu, L. Shen, and G. Sun, “Squeeze-and-excitation networks,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pp. 7132–7141, 2018.
- [5] A. Vaswani *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [6] J. Duarte *et al.*, “Fast inference of deep neural networks in FPGAs for particle physics,” *J. Instrum.*, vol. 13, no. 07, p. P07027, 2018.

- [7] F. Fahim *et al.*, “hls4ml: An open-source codesign workflow for implementing machine learning in FPGAs,” in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 2021.
- [8] T. Aarrestad *et al.*, “Fast convolutional neural networks on FPGAs with hls4ml,” *Mach. Learn.: Sci. Technol.*, vol. 2, no. 4, p. 045015, 2021.
- [9] C. N. Coelho Jr. *et al.*, “Automatic heterogeneous quantization of deep neural networks for low-latency inference on the edge for particle detectors,” *Nature Machine Intelligence*, vol. 3, pp. 675–686, 2021.
- [10] K. Guo *et al.*, “A survey of FPGA-based neural network inference accelerators,” *ACM Trans. Reconfigurable Technol. Syst.*, vol. 12, no. 1, pp. 1–26, 2019.
- [11] Y. Umuroglu *et al.*, “FINN: A framework for fast, scalable binarized neural network inference,” in *Proc. ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays (FPGA)*, pp. 65–74, 2017.
- [12] M. Blott *et al.*, “FINN-R: An end-to-end deep-learning framework for fast exploration of quantized neural networks,” *ACM Trans. Reconfigurable Technol. Syst.*, vol. 11, no. 3, pp. 1–23, 2018.
- [13] H. Ismail Fawaz, G. Forestier, J. Weber, L. Idoumghar, and P.-A. Muller, “Deep learning for time series classification: A review,” *Data Mining Knowl. Discov.*, vol. 33, no. 4, pp. 917–963, 2019.
- [14] Z. Wang, W. Yan, and T. Oates, “Time series classification from scratch with deep neural networks: A strong baseline,” in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, pp. 1578–1585, 2017.
- [15] M. Kachuee, S. Fazeli, and M. Sarrafzadeh, “ECG heartbeat classification: A deep transferable representation,” in *Proc. IEEE Int. Conf. Healthcare Informatics (ICHI)*, pp. 443–444, 2018.
- [16] H. A. Dau *et al.*, “The UCR time series archive,” *IEEE/CAA J. Automatica Sinica*, vol. 6, no. 6, pp. 1293–1305, 2019.
- [17] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proc. Int. Conf. Machine Learning (ICML)*, pp. 448–456, 2015.
- [18] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pp. 770–778, 2016.

- [19] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Int. Conf. Learn. Representations (ICLR)*, 2015.